```
orders = Order.checks.last_n_days(7)
```

In addition to making your application code easier to write and easier to read, scopes can make your code more efficient. The previous statement, for example, is implemented as a single SQL query.

`ActiveRecord::Relation` objects are equivalent to an anonymous scope:

```
in_house = Order.where('email LIKE "%@pragprog.com"')
```

Of course, relations can also be combined:

```
in_house.checks.last_n_days(7)
```

Scopes aren't limited to where conditions; we can do pretty much anything we can do in a method call: `limit`, `order`, `join`, and so on. Just be aware that Rails doesn't know how to handle multiple `order` or `limit` clauses, so be sure to use these only once per call chain.

In nearly every case, the methods we've been describing are sufficient. But Rails isn't satisfied with only being able to handle nearly every case, so for cases that require a human-crafted query, there's an API for that too.

Writing Our Own SQL

Each of the methods we've been looking at contributes to the construction of a full SQL query string. The method `find_by_sql` lets our application take full control. It accepts a single parameter containing a SQL `select` statement (or an array containing SQL and placeholder values, as for `find`) and returns an array of model objects (that's potentially empty) from the result set. The attributes in these models will be set from the columns returned by the query. We'd normally use the `select *` form to return all columns for a table, but this isn't required:

rails80/e1/ar/find_examples.rb